

TALLER 3

INTEGRANTES:

Jeison Steven Guio Varón

Laura Victoria Flórez Forero

Matemáticas Discretas I

29/07/2026

Universidad Nacional de Colombia

1. Cifrado César: romper un mensaje antiguo

1. ¿Qué problema resuelve el programa?

El programa facilita el cifrado y descifrado de mensajes usando el Cifrado César. Ya que este cifrado consiste en “mover” la posición de cada letra en un mensaje un cierto número de posiciones ‘k’ el proceso de mover cada letra de una frase o larga puede tomar algo de tiempo, pero con el programa este proceso es instantáneo. Además, para los casos en los que el valor de ‘k’ se desconoce el programa permite un fácil descifrado de fuerza bruta mostrando todos los mensajes resultantes de descifrar con todos los valores de ‘k’ posibles (25 posibles k, sin contar el mensaje original, $k = 0$, valor proveniente del número de “posiciones” de letras en el abecedario sin la ñ), permitiendo que el usuario busque el mensaje con sentido para el descifrado

2. ¿Qué idea matemática usa?

El programa se centra en la aplicación del Cifrado César, el cual usa el desplazamiento de las letras en el alfabeto un determinado ‘k’ número de veces con el fin de poder cifrar y descifrar mensajes fácilmente. Para la poder ejecutar el Cifrado César fácilmente en el programa se usó la aritmética modular en el alfabeto de 26 letras, con $N_posicion = (posicion + k) \% 26$ para cifrar y $N_posicion = (posicion - k) \% 26$ para descifrar letra por letra.

3. ¿Cómo se ejecuta?

Ya que el programa no necesita de ninguna biblioteca, basta con ejecutar el archivo “1_CifradoCesar.py” en cualquier editor de código, elegir la opción de lo que el usuario desee hacer e ingresar el texto a cifrar o descifrar

4. ¿Qué pruebas hicieron?

Prueba de cifrado de texto con la frase “HOLA UNAL” y $k = 3$, salida: “KROD XQDO”.

```

.*+° ¡Bienvenido al programa de Cifrado César! °+*.

|| En el cifrado César se desplaza cada letra de un texto cifrado un cierto número de posiciones 'k': ||
|| - Si se quiere cifrar un texto, el cifrado mueve la posición de cada letra k veces hacia adelante. ||
|| - Si se conoce k, el descifrado mueve la posición de cada letra k veces hacia atrás. ||
|| - También se puede realizar un descifrado con fuerza bruta si se desconoce k: ||
||   Es decir, el texto se mueve un total de 25 posibles k, y se revisa cuál de ellos tiene sentido. ||
|| El programa conserva espacios, mayúsculas, signos de puntuación y números. ||

=====
Por favor elija lo que desea hacer:
1. Cifrar un texto con desplazamiento k
2. Descifrar un texto (conociendo k)
3. Descifrar probando todos los desplazamientos k (si no se conoce k)
4. Salir del programa
=====

Ingrese el número de la opción (1, 2, 3 o 4): 1

.*+° Cifrado de texto °+*.

Ingrese el texto a cifrar: HOLA UNAL
Ingrese el desplazamiento (k) deseado: 3

-----
Texto original:   HOLA UNAL
Desplazamiento k: 3
Texto cifrado:   KROD XQDO
-----

```

Prueba de descifrado con el mismo cifrado anterior “KROD XQDO” y $k = 3$ para confirmar que el descifrado y el cifrado son coherentes, salida: “HOLA UNAL”.

```

=====
Por favor elija lo que desea hacer:
1. Cifrar un texto con desplazamiento k
2. Descifrar un texto (conociendo k)
3. Descifrar probando todos los desplazamientos k (si no se conoce k)
4. Salir del programa
=====

Ingrese el número de la opción (1, 2, 3 o 4): 2

.*+° Descifrado de texto °+*.
Ingrese el texto cifrado: KROD XQDO
Ingrese el desplazamiento usado para cifrar (k): 3

-----
Texto cifrado:   KROD XQDO
Desplazamiento: 3
Texto descifrado: HOLA UNAL
-----

```

Prueba de descifrado sin saber k (fuerza bruta) con la misma frase cifrada “KROD XQDO”, como podemos comprobar en $k = 3$ aparece la frase descifrada correctamente “HOLA UNAL”.

```

=====
Ingrese el número de la opción (1, 2, 3 o 4): 3

.*+º Descifrado con fuerza bruta º+*.
Ingrese el texto cifrado: KROD XQDO

=====
Probando todos los desplazamientos (0 a 25):
=====
0 | KROD XQDO
1 | JQNC WPCN
2 | IPMB VOBM
3 | HOLA UNAL
4 | GNKZ TMZK
5 | FMDY SLYJ
6 | ELIX RKXI
7 | DKHW QJWH
8 | CJGV PIVG
9 | BIFU OHUF
10 | AHET NGTE
11 | ZGDS MFSD
12 | YFCR LERC
13 | XEBQ KDQB
14 | WDAP JCPA
15 | VCZO IBOZ
16 | UBYN HANY
17 | TAXM GZMX
18 | SZWL FYLW
19 | RYVK EXKV
20 | QXUJ DWJU
21 | PWIT CVIT
22 | OVSH BUHS
23 | NURG ATGR
24 | MTQF ZSFQ
25 | LSPE YREP

-----
-> Busca entre los resultados el que parezca un mensaje con sentido:
    Ese será el mensaje descifrado.
=====

```

Prueba de cifrado de una frase con símbolos y mayúsculas para evidenciar que el programa conserva, espacios, signos de puntuación y otros caracteres. Además de que funciona con mayúsculas y minúsculas, manteniendo la coherencia en el cifrado.

```

=====
Ingrese el número de la opción (1, 2, 3 o 4): 1

.*+º Cifrado de texto º+*.

Ingrese el texto a cifrar: ¡Hola Profe! :)
Ingrese el desplazamiento (k) deseado: 7

-----
Texto original:  ¡Hola Profe! :)
Desplazamiento k:  7
Texto cifrado:  ¡Ovsh Wymml! :)
=====

```

5. ¿Qué limitaciones tiene la solución?

Las limitaciones de la solución vienen principalmente de la naturaleza del Cifrado César: ya que el proceso de cifrado y descifrado de mensajes con este cifrado es simple, un humano podría descifrar por fuerza bruta cualquier mensaje, por ello, no es un tipo de cifrado seguro, mucho

menos para un contexto en el que programas simples como este pueden realizar cifrados y descifrados en segundos.

Específicamente en mi programa, el descifrado con fuerza bruta podría tener más de un mensaje resultante con sentido y los caracteres no pertenecientes al alfabeto se mantienen y no se cifran de ninguna manera.

6. Expliquen por qué el descifrado usa el desplazamiento contrario y por qué el ataque de fuerza bruta es posible en este cifrado.

El descifrado usa el desplazamiento contrario porque es la operación inversa del cifrado. Si para cifrar se sumó una cantidad 'k' a la posición de cada letra en el alfabeto ($posicion + k$), para descifrar se debe restar esa misma cantidad ($posicion - k$), devolviendo así cada letra a su posición original.

El ataque de fuerza bruta es posible en el Cifrado César porque las posibilidades de cifrado son pequeñas: solo existen 26 desplazamientos posibles ($k = 0, 1, \dots, 24, 25$). Esto significa que se pueden probar todas las opciones en cuestión de segundos, incluso manualmente, y el resultado correcto será el único que tenga sentido como mensaje. Esta es precisamente la debilidad principal del cifrado y la razón por la que no se usa en aplicaciones de seguridad real.

2. RSA de juguete: llaves, cifrado y descifrado

1. ¿Qué problema resuelve el programa?

El programa implementa una versión educativa de RSA, un sistema criptográfico asimétrico que permite cifrar y descifrar mensajes usando dos llaves diferentes: una pública para cifrar y una privada para descifrar. A diferencia del Cifrado César, RSA es un sistema de llave pública donde cualquiera puede cifrar un mensaje, pero solo quien tiene la llave privada puede descifrarlo. El programa recibe dos números primos p y q , y un exponente público e , calcula automáticamente todas las partes necesarias (n , $\phi(n)$, d) y permite cifrar y descifrar un mensaje numérico, mostrando todo el proceso paso a paso para que sea fácil de entender.

2. ¿Qué idea matemática usa?

El programa utiliza la aritmética modular y propiedades de los números primos. Primero se calcula $n = p * q$ (donde p y q son primos), luego $\phi(n) = (p-1) * (q-1)$ que es la función totiente de Euler, se verifica que $\text{mcd}(e, \phi(n)) = 1$ para asegurar que e sea válido y finalmente se calcula d como el inverso modular de e módulo $\phi(n)$ usando el algoritmo de Euclides extendido. Para el cifrado se usa $C = M^e \bmod n$ y para el descifrado $M = C^d \bmod n$, lo cual funciona porque $(M^e)^d \equiv M^{ed} \equiv M^1 \equiv M \pmod{n}$, gracias a que $ed \equiv 1 \pmod{\phi(n)}$. La seguridad de RSA se basa en la dificultad de factorizar n en sus factores primos p y q , y en esta versión de juguete usamos números pequeños para simplificarlo.

3. ¿Cómo se ejecuta?

Ya que el programa no necesita de ninguna biblioteca externa, basta con ejecutar el archivo "2_RSA.py" en cualquier editor de código. El programa solicitará ingresar los números primos p y q , el exponente público e , y luego el mensaje a cifrar, y también verifica que e sea válido mostrando todo el proceso paso a paso.

4. ¿Qué pruebas hicieron?

Prueba del caso obligatorio del taller con $p = 61$, $q = 53$, $e = 17$, $M = 65$, obteniendo $n = 3233$, $\phi(n) = 3120$, $d = 2753$, $C = 2790$, y al descifrar regresó $M = 65$, confirmando que el programa funciona correctamente.

```

.*+º ¡Bienvenido al programa de RSA de Juguetes! º+*.

|| RSA es un sistema criptográfico basado en aritmética modular. ||
|| Se usa una versión pequeña para entender la idea matemática. ||
|| - Se reciben dos números primos p, q y un exponente público e. ||
|| - Se calcula n = p*q y  $\phi(n) = (p-1)*(q-1)$ . ||
|| - Se calcula d, el inverso modular de e módulo  $\phi(n)$ . ||
|| - Con esto se puede cifrar y descifrar mensajes. ||

=====
Por favor elija lo que desea hacer:
1. Generar llaves y cifrar/descifrar un mensaje
2. Salir del programa
=====

Ingrese el número de la opción (1 o 2): 1

.*+º Generación de llaves RSA º+*.

Ingrese el número primo p: 61
Ingrese el número primo q: 53
Ingrese el exponente público e: 17

-----
n = p * q = 61 * 53 = 3233
 $\phi(n) = (p-1)*(q-1) = 60 * 52 = 3120$ 
d = inverso de 17 módulo 3120 = 2753
-----

Ingrese el mensaje a cifrar (número entero): 65

Cifrado: C =  $65^{17} \bmod 3233 = 2790$ 
Descifrado: M =  $2790^{2753} \bmod 3233 = 65$ 

-----
¡El cifrado y descifrado funcionaron correctamente!
-----

```

Prueba con $p = 7$, $q = 5$, y $e = 3$, obteniendo que para esos valores 'e' no es válido.

```

.*+º Generación de llaves RSA º+*.

Ingrese el número primo p: 7
Ingrese el número primo q: 5
Ingrese el exponente público e: 3

-----
n = p * q = 7 * 5 = 35
 $\phi(n) = (p-1)*(q-1) = 6 * 4 = 24$ 

e = 3 NO es válido porque no es coprimo con  $\phi(n) = 24$ 
El MCD(e,  $\phi(n)$ ) debe ser 1. Intente con otro valor de e.
-----

```

Prueba con números no válidos: el programa detecta que no es un número primo o mayor o igual a 2. Además usando p, q, e y el mensaje a descifrar = 2 el programa devuelve que el mensaje descifrado no coincide con el original.

```

Ingrese el número de la opción (1 o 2): 1

.*+º Generación de llaves RSA º+*.

Ingrese el número primo p: 10
10 no es un número primo. Por favor ingrese un número primo.

Ingrese el número primo p: -3
El número debe ser mayor o igual a 2.

Ingrese el número primo p: 1
El número debe ser mayor o igual a 2.

Ingrese el número primo p: 2
Ingrese el número primo q: 2
Ingrese el exponente público e: 2

-----
n = p * q = 2 * 2 = 4

 $\phi(n) = (p-1)*(q-1) = 1 * 1 = 1$ 

d = inverso de 2 módulo 1 = 0
-----

Ingrese el mensaje a cifrar (número entero): 2

Cifrado: C =  $2^2 \bmod 4 = 0$ 
Descifrado: M =  $0^0 \bmod 4 = 1$ 

-----
El mensaje descifrado no coincide con el original.
-----

```

5. ¿Qué limitaciones tiene la solución?

Las limitaciones principales son que al usar números pequeños el sistema no es seguro para uso real, ya que RSA real usa primos de cientos de dígitos para garantizar la seguridad. Además, el programa solo cifra números enteros y no texto directamente, por lo que para cifrar texto real se necesitaría una codificación previa. La validación de primos funciona bien para números pequeños pero para números muy grandes sería extremadamente lenta. También existe la limitación de que el mensaje M debe ser menor que n, porque si $M \geq n$ el cifrado no funciona correctamente ya que la operación $M \bmod n$ perdería información, aunque para números pequeños esto no suele ser un problema.

6. Expliquen el papel de los primos, del inverso modular y de la congruencia en el proceso.

Los números primos p y q son la base de todo el sistema RSA, ya que su producto n forma parte de la llave pública y la función $\phi(n) = (p-1)(q-1)$ solo se puede calcular fácilmente si se conocen p y q, y si un atacante quisiera descifrar un mensaje sin la llave privada necesitaría factorizar n

en p y q , lo cual es computacionalmente inviable para números grandes, y de ahí viene precisamente la seguridad del sistema.

El inverso modular d es la llave privada y se calcula como $d \equiv e^{-1} \pmod{\phi(n)}$, es decir, $e * d \equiv 1 \pmod{\phi(n)}$, lo que permite que el cifrado y descifrado sean operaciones inversas entre sí porque al cifrar con e y luego descifrar con d se recupera el mensaje original ya que $(M^e)^d \equiv M^{(e*d)} \equiv M^1 \equiv M \pmod{n}$, y sin d es prácticamente imposible recuperar el mensaje.

La congruencia es la base de toda la aritmética de RSA, donde $C \equiv M^e \pmod{n}$ significa que C es el residuo de dividir M^e entre n , y esta operación es fácil de calcular pero difícil de invertir sin conocer d , lo que da la seguridad al sistema, además de que permite que el proceso sea cíclico y que los números se mantengan en un rango manejable ya que todos los cálculos se realizan módulo n .

3. MPC básico: calcular un promedio sin mostrar los datos

1. ¿Qué problema resuelve el programa?

El programa simula un protocolo de computación multipartita segura para calcular el promedio de las notas de un grupo de estudiantes sin revelar los datos individuales de cada uno. Esto es útil en situaciones donde se quiere conocer la suma total y el promedio de un conjunto de datos, pero que por privacidad o confidencialidad no se pueden compartir los valores individuales. Cada nota se divide en tres partes aleatorias y se distribuye entre tres servidores diferentes, de manera que ningún servidor por sí solo puede reconstruir ninguna nota original, y al final solo se obtiene la suma total y el promedio.

2. ¿Qué idea matemática usa?

El programa utiliza la aritmética modular para ocultar los datos individuales. Cada nota x se escribe como $x \equiv s_1 + s_2 + s_3 \pmod{M}$, donde s_1 , s_2 y s_3 son tres partes aleatorias generadas de tal forma que su suma módulo M sea igual a la nota original. Esto se logra generando s_1 y s_2 de forma aleatoria y calculando $s_3 = (x - s_1 - s_2) \pmod{M}$. De esta manera, cada parte por sí sola parece un número aleatorio sin relación con la nota original, y solo al sumar las tres partes se puede recuperar el valor original. Al final, los servidores entregan sus sumas parciales y se reconstruye la suma total con $(\text{servidor1} + \text{servidor2} + \text{servidor3}) \pmod{M}$.

3. ¿Cómo se ejecuta?

El programa usa la biblioteca `random` para generar números aleatorios, pero al ser parte de la biblioteca estándar de Python no necesita instalación adicional. Basta con ejecutar el archivo `"3_MPC.py"` en cualquier editor de código, elegir la opción 1 para calcular el promedio seguro, ingresar la cantidad de notas y cada una de las notas entre 0 y 50, y el programa mostrará únicamente la suma total y el promedio calculados de forma segura.

4. ¿Qué pruebas hicieron?

Prueba del caso ejemplo del taller, con 4 notas: 40, 35, 50 y 25. Se obtuvo como suma total 150 y como promedio 37.50 sin mostrar nada más.


```
.+° ¡Bienvenido al programa de MPC Básico! °+*.

|| MPC (Computación Multipartita Segura) permite calcular un promedio ||
|| sin revelar los datos individuales de cada estudiante. ||
|| - Cada nota se divide en 3 partes aleatorias módulo M. ||
|| - Cada servidor recibe una parte de cada nota. ||
|| - Al final se reconstruye únicamente la suma total y el promedio. ||
|| - Ningún servidor por sí solo puede conocer las notas originales. ||

=====
Por favor elija lo que desea hacer:
1. Calcular promedio de notas de forma segura
2. Salir del programa
=====

Ingrese el número de la opción (1 o 2): 1

.+° Cálculo de promedio seguro °+*.

¿Cuántas notas desea ingresar? 4
Ingrese la nota 1 (0-50): 40
Ingrese la nota 2 (0-50): 35
Ingrese la nota 3 (0-50): 50
Ingrese la nota 4 (0-50): 25

-----
Suma total: 150
Promedio: 37.50
-----
```

Prueba con 6 notas de 0 a 5. Se obtuvo como suma total 15 y como promedio 2.50.

```
Ingrese el número de la opción (1 o 2): 1

.+° Cálculo de promedio seguro °+*.

¿Cuántas notas desea ingresar? 6
Ingrese la nota 1 (0-50): 0
Ingrese la nota 2 (0-50): 1
Ingrese la nota 3 (0-50): 2
Ingrese la nota 4 (0-50): 3
Ingrese la nota 5 (0-50): 4
Ingrese la nota 6 (0-50): 5

-----
Suma total: 15
Promedio: 2.50
-----
```

Prueba ingresando datos inválidos en el número de notas a ingresar y en el valor de la nota. El programa pide un número positivo tanto para el número de notas a ingresar como para el valor de la nota en sí (0 – 50). Posteriormente realiza el promedio correctamente.

```
Ingrese el número de la opción (1 o 2): 1
```

```
.*+° Cálculo de promedio seguro °+*.
```

```
¿Cuántas notas desea ingresar? 0
```

```
Por favor ingrese un número entero válido.
```

```
¿Cuántas notas desea ingresar? -2
```

```
Debe ingresar al menos una nota.
```

```
¿Cuántas notas desea ingresar? 0
```

```
Debe ingresar al menos una nota.
```

```
¿Cuántas notas desea ingresar? 3
```

```
Ingrese la nota 1 (0-50): -2
```

```
La nota debe estar entre 0 y 50.
```

```
Ingrese la nota 1 (0-50): 0
```

```
Ingrese la nota 2 (0-50): 50
```

```
Ingrese la nota 3 (0-50): 25
```

```
-----  
Suma total: 75
```

```
Promedio: 25.00  
-----
```

5. ¿Qué limitaciones tiene la solución?

Las limitaciones principales son que esta es una simulación educativa y no un protocolo de seguridad industrial real, por lo que no considera aspectos como autenticación de servidores, canales seguros de comunicación o protección contra servidores maliciosos que podrían mentir sobre sus sumas parciales. Además, el programa asume que todos los servidores son honestos y siguen el protocolo correctamente, lo cual en la vida real no siempre es cierto. El módulo $M = 1000007$ es suficientemente grande para este propósito educativo, pero en aplicaciones reales se necesitarían medidas adicionales de seguridad. También se debe tener en cuenta que el programa muestra las notas originales durante la ejecución para fines de verificación, aunque en un protocolo real estas nunca deberían ser visibles.

6. Expliquen con un ejemplo pequeño cómo una nota x se puede escribir como $x \equiv s_1 + s_2 + s_3 \pmod{M}$ sin que una sola parte revele x .

Supongamos que tenemos una nota $x = 42$ y usamos un módulo $M = 100$. Para dividir esta nota en tres partes, generamos dos números aleatorios, por ejemplo $s_1 = 35$ y $s_2 = 87$, y luego calculamos $s_3 = (42 - 35 - 87) \bmod 100 = (42 - 122) \bmod 100 = (-80) \bmod 100 = 20$. Ahora tenemos las tres partes $s_1 = 35$, $s_2 = 87$, $s_3 = 20$, y al sumarlas tenemos $35 + 87 + 20 = 142$, y $142 \bmod 100 = 42$, que es la nota original. Cada parte por separado (35, 87 o 20) no revela absolutamente nada sobre la nota original 42, porque son números aleatorios que podrían

corresponder a cualquier valor. Incluso si un servidor tiene la parte $s_1 = 35$, no puede saber si la nota original era 42, 135, 235 o cualquier otro número que sea congruente con 42 módulo 100, y la aleatoriedad de las partes hace que sea imposible deducir la nota original a partir de una sola parte.

4. Ruta más corta: una ciudad como grafo

1. ¿Qué problema resuelve el programa?

El programa resuelve el problema de si al tener un grafo ponderado con pesos en sus vértices, de manera automática el saber qué ruta es la más conveniente o el más óptimo, ya que encontrar la ruta que tenga menos valor hace que en sistemas reales o en aplicaciones reales se de respuesta rápida a la pregunta de qué rutas resultan en un menor costo ya sea de tiempo o dinero

2. ¿Qué idea matemática usa?

El programa se basa en la teoría de grafos, donde los vértices representan lugares y las aristas representan las conexiones entre ellos. Para encontrar el camino óptimo utiliza el algoritmo de Dijkstra, el cual compara las distancias posibles entre los vértices y selecciona siempre el camino de menor costo siempre que los vértices no tengan pesos negativos, ya que asume que una vez encuentra la menor distancia hacia un vértice, esta ya es la mínima posible y no necesitará actualizarse. Si existieran pesos negativos, podría aparecer posteriormente un camino con un costo menor hacia un vértice ya procesado, produciendo resultados incorrectos. Por ello, Dijkstra solo garantiza encontrar la ruta óptima cuando todos los pesos del grafo son mayores o iguales a cero.

3. ¿Cómo se ejecuta?

Se necesita de un dispositivo el cual tenga la capacidad de correr programas con el lenguaje de Python, así que se requiere tener el lenguaje de programación instalado, y al mismo tiempo solo se necesita un editor de código de confianza que tenga la capacidad de leer archivos en Python. El usuario ingresa el vértice de origen y el de destino. El algoritmo verifica que ambos existan, calcula la ruta más corta mediante Dijkstra y finalmente muestra la secuencia de vértices que forman el camino y la distancia total recorrida.

4. ¿Qué pruebas hicieron?

Se realizaron pruebas utilizando un grafo de ejemplo con más de ocho vértices y doce aristas. Se verificó que el algoritmo encontrara correctamente la ruta de menor costo entre diferentes pares de vértices. Además, se probaron casos en los que el vértice de origen o destino no existía y situaciones en las que no había un camino disponible entre ambos, comprobando que el programa respondiera adecuadamente.

```
Por favor, teniendo en cuenta el grafo mostrado, ingrese el nombre del vertice de inicio y de destino a continuacion:
```

```
Vertice de inicio: Centro
```

```
Vertice de destino: Hospital
```

```
Según los datos obtenidos y la implementación del algoritmo de Dijkstra se tiene que la ruta mas corta, junto con su peso designado es:
```

```
Ruta: Centro -> Hospital
```

```
Distancia total: 4
```

```
Por favor, teniendo en cuenta el grafo mostrado, ingrese el nombre del vertice de inicio y de destino a continuacion:
Vertice de inicio: Estadio
Vertice de destino: Calle26

Según los datos obtenidos y la implementación del algoritmo de Dijkstra se tiene que la ruta mas corta, junto con su peso designado es:

Ruta: Estadio -> Hospital -> Centro -> Calle26
Distancia total: 13

Por favor, teniendo en cuenta el grafo mostrado, ingrese el nombre del vertice de inicio y de destino a continuacion:
Vertice de inicio: Universidad
Vertice de destino: Portal

Según los datos obtenidos y la implementación del algoritmo de Dijkstra se tiene que la ruta mas corta, junto con su peso designado es:

Ruta: Universidad -> Terminal -> Portal
Distancia total: 9
```

5. ¿Qué limitaciones tiene la solución?

La principal limitación es que el algoritmo de Dijkstra solo funciona correctamente con grafos cuyos pesos sean no negativos. Además, la implementación realizada busca manualmente el vértice con la menor distancia en cada iteración, por lo que resulta menos eficiente si se usara en grafos muy grandes con muchos vértices y aristas.

5. Cierre de una estación: medir el impacto en la red

1. ¿Qué problema resuelve el programa?

El programa resuelve el problema de encontrar la ruta más corta entre diferentes pares de estaciones dentro de una red representada mediante un grafo, tal y como se hizo en el ejercicio 4. Además, simula el cierre de una estación eliminándola del grafo junto con todas sus conexiones, para comparar cómo cambian las distancias entre los distintos puntos de la red antes y después de dicho cierre, algo que puede pasar perfectamente en un sistema real, ya que en un sistema dinámico en donde un vértice puede llegar a fallar siempre será necesario realizar un análisis de cómo afecta esto a todo el grafo

2. ¿Qué idea matemática usa?

Como en el ejercicio anterior: El programa se basa en la teoría de grafos, donde los vértices representan lugares y las aristas representan las conexiones entre ellos. Para encontrar el camino óptimo utiliza el algoritmo de Dijkstra, el cual compara las distancias posibles entre los vértices y selecciona siempre el camino de menor costo siempre que los vértices no tengan pesos negativos, ya que asume que una vez encuentra la menor distancia hacia un vértice, esta ya es la mínima posible y no necesitará actualizarse.

3. ¿Cómo se ejecuta?

Se necesita un editor de código capaz de ejecutar Python. El problema sigue la lógica del punto 4, la diferencia es que el vértice de origen y de destino ya están prefijados, y el usuario solo tiene que ingresar el vértice que desea eliminar siguiente esa configuración de vértices ya establecidos, observando qué cambios produce en las conexiones de esos pares de vértices respecto a su distancia o si es posible llegar a ellos

4. ¿Qué pruebas hicieron?

Las pruebas se hicieron usando el mismo grafo de ocho vértices y doce aristas, y en base a ese grafo y a los pares de vértices que se buscaban analizar, se probó si los vértices aumentaban el valor de su distancia al eliminar un vértice que se le pide al usuario ingresar.

```
=====
Ingrese el nombre del vertice que desea eliminar, que haga parte del grafo dado: Hospital
Se eliminó el vértice 'Hospital' junto con todas sus conexiones.
```

```
Error: el vértice 'Hospital' no existe en el grafo
```

```
Aplicando el algoritmo de Dijkstra para calcular la menor distancia de dos vertices antes y despues
de eliminar un vertice (en este caso el vertice Hospital) se obtiene los siguientes resultados
```

Origen	Destino	Antes	Después

Portal	Hospital	12	No existe
Portal	Estadio	14	20
Calle26	Universidad	13	13
Museo	Terminal	12	12
Parque	Centro	8	8

```
=====
Ingrese el nombre del vertice que desea eliminar, que haga parte del grafo dado: Terminal
Se eliminó el vértice 'Terminal' junto con todas sus conexiones.
```

```
Error: el vértice 'Terminal' no existe en el grafo
```

```
Aplicando el algoritmo de Dijkstra para calcular la menor distancia de dos vertices antes y despues
de eliminar un vertice (en este caso el vertice Hospital) se obtiene los siguientes resultados
```

Origen	Destino	Antes	Después

Portal	Hospital	12	15
Portal	Estadio	14	17
Calle26	Universidad	13	14
Museo	Terminal	12	No existe
Parque	Centro	8	8

```
=====
Ingrese el nombre del vertice que desea eliminar, que haga parte del grafo dado: Estadio
Se eliminó el vértice 'Estadio' junto con todas sus conexiones.
```

```
Error: el vértice 'Estadio' no existe en el grafo
```

```
Aplicando el algoritmo de Dijkstra para calcular la menor distancia de dos vertices antes y despues
de eliminar un vertice (en este caso el vertice Hospital) se obtiene los siguientes resultados
```

Origen	Destino	Antes	Después

Portal	Hospital	12	12
Portal	Estadio	14	No existe
Calle26	Universidad	13	13
Museo	Terminal	12	12
Parque	Centro	8	8

5. ¿Qué limitaciones tiene la solución?

La principal limitación es que el algoritmo de Dijkstra solo funciona correctamente cuando todas las aristas tienen pesos no negativos. Además, la implementación utilizada busca el siguiente

vértice recorriendo una lista de vértices no visitados, lo que hace que el tiempo de ejecución aumente en grafos de gran tamaño. Por último, el programa trabaja sobre un grafo definido manualmente y solo permite simular el cierre de una estación a la vez, por lo que no contempla cambios más complejos en la red

6. Coloreo de grafos: organizar exámenes sin choques

1. ¿Qué problema resuelve el programa?

El programa resuelve el problema de asignar horarios de examen a diferentes materias evitando que dos materias con estudiantes en común tengan examen al mismo tiempo. Para ello se representa las materias y sus conflictos mediante un grafo y asigna un color, que representa un mismo horario, a cada materia, de forma que dos materias conectadas nunca compartan el mismo color. Problema que se podría aplicar a múltiples otros problemas que requieran diferencias cada vértice de sus vecinos

2. ¿Qué idea matemática usa?

El programa usa la teoría de grafos, específicamente el coloreo de grafos. Cada vértice representa una materia y cada arista representa un conflicto entre dos materias porque comparten estudiantes. El algoritmo aplica un método voraz, asignando a cada vértice el primer color disponible que no esté siendo utilizado por sus vértices vecinos. Así, cada color representa una franja horaria distinta para los exámenes.

3. ¿Cómo se ejecuta?

El programa se ejecuta usando Python, definiendo primero el grafo de conflictos entre las materias, el cual en este caso se definió directamente para ahorrar el tiempo de ingreso de datos al usuario ya que tienen que ser mínimo 10 vértices. Luego recorre cada vértice y analiza los colores que ya tienen sus vecinos para asignarle el primer color disponible. Una vez coloreado todo el grafo, muestra el color asignado a cada materia, el número total de colores utilizados y las materias que pertenecen a cada color.

4. ¿Qué pruebas hicieron?

Se realizaron pruebas utilizando distintos grafos de al menos diez vértices, cambiando las conexiones entre las materias para simular diferentes niveles de conflicto.

```
Matematicas -> Fisica, Programacion, Algebra,  
Fisica -> Matematicas, Quimica, BasesDatos,  
Programacion -> Matematicas, BasesDatos, Redes,  
Algebra -> Matematicas, Estadistica,  
Quimica -> Fisica, Biologia,  
BasesDatos -> Fisica, Programacion, Redes,  
Redes -> Programacion, BasesDatos, Electronica,  
Biologia -> Quimica,  
Electronica -> Redes, Estadistica,  
Estadistica -> Algebra, Electronica,  
Color asignado a cada materia
```

```
Matematicas      -> Color 1  
Fisica           -> Color 2  
Programacion     -> Color 2  
Algebra          -> Color 2  
Quimica          -> Color 1  
BasesDatos       -> Color 1  
Redes            -> Color 3  
Biologia         -> Color 2  
Electronica      -> Color 1  
Estadistica      -> Color 3
```

Cantidad de colores utilizados: 3

Materias en cada color:

Color 1:

- Matematicas
- Quimica
- BasesDatos
- Electronica

Color 2:

- Fisica
- Programacion
- Algebra
- Biologia

Color 3:

- Redes
- Estadistica

```
Matematicas -> Fisica, Programacion, Algebra, Quimica,  
Fisica -> Matematicas, Programacion, BasesDatos,  
Programacion -> Matematicas, Fisica, Redes,  
Algebra -> Matematicas, Estadistica, Electronica,  
Quimica -> Matematicas, Biologia, BasesDatos,  
BasesDatos -> Fisica, Quimica, Redes,  
Redes -> Programacion, BasesDatos, Electronica,  
Biologia -> Quimica, Estadistica,  
Electronica -> Algebra, Redes, Estadistica,  
Estadistica -> Algebra, Biologia, Electronica,  
Color asignado a cada materia
```

```
Matematicas    -> Color 1  
Fisica         -> Color 2  
Programacion   -> Color 3  
Algebra        -> Color 2  
Quimica        -> Color 2  
BasesDatos     -> Color 1  
Redes          -> Color 2  
Biologia       -> Color 1  
Electronica    -> Color 1  
Estadistica    -> Color 3
```

Cantidad de colores utilizados: 3

Materias en cada color:

Color 1:

- Matematicas
- BasesDatos
- Biologia
- Electronica

Color 2:

- Fisica
- Algebra
- Quimica
- Redes

Color 3:

- Programacion
- Estadistica


```
Matematicas -> Fisica, Programacion,  
Fisica -> Matematicas, Quimica, BasesDatos,  
Programacion -> Matematicas, Redes,  
Algebra -> Estadistica, BasesDatos,  
Quimica -> Fisica, Biologia,  
BasesDatos -> Fisica, Algebra, Electronica,  
Redes -> Programacion, Electronica,  
Biologia -> Quimica, Estadistica,  
Electronica -> BasesDatos, Redes,  
Estadistica -> Algebra, Biologia,  
Color asignado a cada materia
```

```
Matematicas    -> Color 1  
Fisica         -> Color 2  
Programacion   -> Color 2  
Algebra        -> Color 1  
Quimica        -> Color 1  
BasesDatos     -> Color 3  
Redes          -> Color 1  
Biologia       -> Color 2  
Electronica    -> Color 2  
Estadistica    -> Color 3
```

Cantidad de colores utilizados: 3

Materias en cada color:

Color 1:

- Matematicas
- Algebra
- Quimica
- Redes

Color 2:

- Fisica
- Programacion
- Biologia
- Electronica

Color 3:

- BasesDatos
- Estadistica

5. ¿Qué limitaciones tiene la solución?

La principal limitación es que el algoritmo voraz no garantiza obtener el número mínimo de colores posible, ya que el resultado depende del orden en que se recorren los vértices del grafo. Además, el programa requiere que el grafo de conflictos esté definido correctamente desde el inicio; si las conexiones no representan adecuadamente los conflictos entre materias, la asignación de horarios no será válida. Sin embargo, el algoritmo siempre genera un coloreo correcto en el que dos materias con conflicto no comparten el mismo horario.

7. Tablas de verdad y circuitos lógicos

1. ¿Qué problema resuelve el programa?

El programa genera tablas de verdad para expresiones lógicas, mostrando todas las combinaciones posibles de entrada y el resultado final de la expresión para cada una, lo cual permite visualizar el comportamiento de una expresión booleana y verificar su equivalencia con otras expresiones. Además, permite evaluar una expresión con valores concretos que el usuario ingresa, lo que ayuda a entender cómo se comporta el circuito para entradas específicas. El programa incluye las tres expresiones del taller y permite al usuario elegir cuál quiere ver o evaluar.

2. ¿Qué idea matemática usa?

El programa utiliza el álgebra de Boole, que es la base de los circuitos lógicos digitales, y trabaja con los conectivos lógicos AND, OR, NOT y XOR, que son las operaciones fundamentales en electrónica digital. Cada expresión se evalúa para todas las combinaciones posibles de las variables de entrada, generando una tabla que muestra el resultado para cada caso, y esta tabla de verdad es exactamente la representación del comportamiento de un circuito lógico, ya que cada fila corresponde a una posible entrada y su salida correspondiente, y la evaluación con valores concretos permite verificar el resultado para entradas específicas sin tener que revisar toda la tabla.

3. ¿Cómo se ejecuta?

El programa no necesita ninguna biblioteca externa, ya que usa únicamente funciones nativas de Python, así que basta con ejecutar el archivo "7_TablasVerdadCircuitos.py" en cualquier editor de código, elegir entre las opciones de generar una tabla de verdad o evaluar una expresión, seleccionar una de las tres expresiones disponibles, y para la tabla se mostrará automáticamente el resultado de todas las combinaciones posibles mientras que para la evaluación se pedirán los valores concretos de cada variable.

4. ¿Qué pruebas hicieron?

Prueba de mostrar la tabla de verdad de la expresión definida en el taller $(A \wedge B) \vee (\neg C)$.

```

.*+° ¡Bienvenido al programa de Tablas de Verdad! °+*.

|| Las tablas de verdad permiten revisar todas las posibilidades ||
|| de una expresión lógica. En electrónica digital, estas expresiones ||
|| se interpretan como circuitos. ||
|| - Se evaluarán las 3 expresiones siguientes: ||
|| 1. (A ∧ B) ∨ (¬C) ||
|| 2. (A XOR B) ∧ C ||
|| 3. (A ∨ B) ∧ (¬A ∨ C) ||

=====
Por favor elija lo que desea hacer:
1. Ver tabla de verdad de una expresión
2. Evaluar una expresión con valores concretos
3. Salir del programa
=====

Ingrese el número de la opción (1, 2 o 3): 1

.*+° Tabla de Verdad °+*.

Expresiones disponibles:
1. (A ∧ B) ∨ (¬C)
2. (A XOR B) ∧ C
3. (A ∨ B) ∧ (¬A ∨ C)

Elija una expresión (1, 2 o 3): 1

-----
Expresión: (A ∧ B) ∨ (¬C)
-----
A | B | C | Resultado
-----
0 | 0 | 0 | 1
0 | 0 | 1 | 0
0 | 1 | 0 | 1
0 | 1 | 1 | 0
1 | 0 | 0 | 1
1 | 0 | 1 | 0
1 | 1 | 0 | 1
1 | 1 | 1 | 1

```

Prueba de evaluar $(A \wedge B) \vee (\neg C)$ con $A = 0$, $B = 1$ y $C = 0$, la salida resultante es 1.

```

Ingrese el número de la opción (1, 2 o 3): 2

.*+º Evaluación de Expresión º+*.

Expresiones disponibles:
1. (A ∧ B) ∨ (¬C)
2. (A XOR B) ∧ C
3. (A ∨ B) ∧ (¬A ∨ C)

Elija una expresión (1, 2 o 3): 1

-----
Evaluando: (A ∧ B) ∨ (¬C)
-----
Ingrese el valor de A (0 o 1): 0
Ingrese el valor de B (0 o 1): 1
Ingrese el valor de C (0 o 1): 0

-----
Resultado: 1
-----

```

Prueba de mostrar la tabla de verdad de otra de las expresiones definidas, en este caso de $(A \vee B) \wedge (\neg A \vee C)$.

```

Ingrese el número de la opción (1, 2 o 3): 1

.*+º Tabla de Verdad º+*.

Expresiones disponibles:
1. (A ∧ B) ∨ (¬C)
2. (A XOR B) ∧ C
3. (A ∨ B) ∧ (¬A ∨ C)

Elija una expresión (1, 2 o 3): 3

-----
Expresión: (A ∨ B) ∧ (¬A ∨ C)
-----
A | B | C | Resultado
-----
0 | 0 | 0 | 0
0 | 0 | 1 | 0
0 | 1 | 0 | 1
0 | 1 | 1 | 1
1 | 0 | 0 | 0
1 | 0 | 1 | 1
1 | 1 | 0 | 0
1 | 1 | 1 | 1

```

5. ¿Qué limitaciones tiene la solución?

El programa está diseñado específicamente para las tres expresiones del taller y usa variables A, B y C, aunque podría extenderse para aceptar expresiones personalizadas y trabajar con variables D si se deseara, pero en su estado actual solo maneja las tres expresiones predefinidas. El uso de `eval()` para evaluar las expresiones, aunque es práctico, podría ser riesgoso si se permitieran expresiones ingresadas por el usuario sin restricciones, pero como las expresiones están predefinidas no hay problema de seguridad. Además, el programa no incluye verificación de equivalencia entre expresiones, solo genera las tablas y las evalúa para entradas concretas.

6. Expliquen cómo se relaciona una tabla de verdad con un circuito lógico.

Una tabla de verdad es la representación abstracta del comportamiento de un circuito lógico, donde cada fila muestra qué salida produce el circuito para cada combinación posible de entradas, y cada columna corresponde a una señal de entrada o salida. En un circuito físico, las variables A, B y C serían cables de entrada que transportan señales de voltaje (0 o 1) y la expresión lógica define cómo se conectan las compuertas (AND, OR, NOT, XOR) para producir la salida, de manera que la tabla de verdad permite predecir exactamente qué salida tendrá el circuito para cualquier combinación de entradas sin necesidad de construirlo físicamente. Por ejemplo, la expresión $(A \wedge B) \vee (\neg C)$ se puede implementar con una compuerta AND para A y B, una compuerta NOT para C, y una compuerta OR para combinar ambas salidas, y la tabla de verdad muestra que el resultado será 1 cuando (A y B sean 1) o (C sea 0).

8. Simplificación booleana: hacer un circuito más barato

1. ¿Qué problema resuelve el programa?

El programa resuelve el problema de simplificar una función booleana expresada mediante sus minterminos, transformándola en una expresión equivalente pero más reducida en forma de suma de productos. Esto permite implementar circuitos lógicos más económicos al reducir el número de compuertas necesarias, lo cual se traduce en un ahorro de componentes físicos y en una mayor eficiencia de los circuitos digitales. Además, el programa verifica automáticamente que la expresión simplificada sea equivalente a la original comparando sus tablas de verdad, garantizando que la simplificación no altera el comportamiento de la función.

2. ¿Qué idea matemática usa?

El programa utiliza el álgebra de Boole y el método de simplificación por agrupación de minterminos adyacentes, que se basa en la propiedad de que dos minterminos que difieren en una sola variable se pueden combinar para eliminar esa variable. La tabla de verdad se genera evaluando todas las combinaciones posibles de las variables de entrada (2^n combinaciones) y marcando con 1 aquellas que corresponden a los minterminos indicados. Para simplificar, el programa convierte los minterminos a su representación binaria y busca pares que difieran en un solo bit, agrupándolos y reemplazando la variable que cambia por un '-' que indica que esa variable no importa en el término resultante. Los términos que no se pueden agrupar se mantienen como están y finalmente todos se combinan en una expresión de suma de productos.

3. ¿Cómo se ejecuta?

Se necesita de un dispositivo con la capacidad de ejecutar programas en Python, por lo que se requiere tener el lenguaje de programación instalado junto con un editor de código de confianza.

El programa solicita al usuario ingresar el número de variables (3 o 4) y los minterminos correspondientes, y luego genera automáticamente la tabla de verdad original, aplica el método de simplificación por agrupación y muestra la expresión simplificada, verificando que sea equivalente a la función original.

4. ¿Qué pruebas hicieron?

Para el punto de simplificación booleana, se realizaron pruebas con funciones de 3 y 4 variables utilizando diferentes conjuntos de minterminos. Se verificó el caso base con la función $\{1, 3, 5, 7\}$ que simplifica a C , el caso con 4 variables $\{0, 2, 5, 7, 8, 10, 13, 15\}$ obteniendo una expresión simplificada válida, el caso extremo con todos los minterminos que simplifica a "1", y el caso sin minterminos que simplifica a "0". También se probaron funciones personalizadas ingresadas por el usuario, incluyendo conjuntos aleatorios y casos con minterminos no consecutivos. En todos los casos, se verificó que la expresión simplificada fuera equivalente a la original comparando las tablas de verdad, confirmando que el método de agrupación por adyacencia funciona correctamente. Además, se validó el manejo de errores al ingresar minterminos fuera del rango permitido y números de variables diferentes a 3 o 4.

=====

Ingrese los datos de la funcion booleana que desea simplificar:

Ingrese el numero de variables (3 o 4): 4

Ingrese los minterminos separados por espacios (0 a 15): 3

Funcion con 4 variables y minterminos [3]

Tabla de verdad original:

Tabla de verdad:

A	B	C	D	Resultado

0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Expresion simplificada: $\neg A \wedge \neg B \wedge C \wedge D$

VERIFICACION: La expresion simplificada es equivalente a la original.

Informacion adicional:

- Numero de variables: 4
- Numero de minterminos: 1
- Numero de combinaciones posibles: 16
- Porcentaje de unos: 6.25%

Ingrese los datos de la funcion booleana que desea simplificar:

Ingrese el numero de variables (3 o 4): 3

Ingrese los minterminos separados por espacios (0 a 7): 7

Funcion con 3 variables y minterminos [7]

Tabla de verdad original:

Tabla de verdad:

A	B	C	Resultado

0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Expresion simplificada: 1

VERIFICACION: La expresion simplificada es equivalente a la original.

Informacion adicional:

- Numero de variables: 3
- Numero de minterminos: 1
- Numero de combinaciones posibles: 8
- Porcentaje de unos: 12.50%

Ingrese los datos de la funcion booleana que desea simplificar:

Ingrese el numero de variables (3 o 4): 4

Ingrese los minterminos separados por espacios (0 a 15): 6

Funcion con 4 variables y minterminos [6]

Tabla de verdad original:

Tabla de verdad:

A	B	C	D	Resultado

0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Expresion simplificada: $\neg A \wedge B \wedge C \wedge \neg D$

VERIFICACION: La expresion simplificada es equivalente a la original.

Informacion adicional:

- Numero de variables: 4
- Numero de minterminos: 1
- Numero de combinaciones posibles: 16
- Porcentaje de unos: 6.25%

5. ¿Qué limitaciones tiene la solución?

La principal limitación es que la implementación utiliza una versión básica de simplificación por agrupación que solo realiza una pasada y no el proceso completo de Quine-McCluskey, por lo que en algunos casos podría no encontrar la simplificación mínima posible. El programa solo funciona para funciones de 3 o 4 variables, lo cual es suficiente para el propósito educativo del taller pero limita su aplicación a casos más complejos. Además, el método de agrupación se

vuelve menos eficiente a medida que aumenta el número de variables debido al crecimiento exponencial de las combinaciones.

6. Expliquen qué significa un mintermino y por qué dos expresiones son equivalentes si tienen la misma tabla de verdad.

Un mintermino es la representación numérica de una combinación específica de valores de las variables de entrada para la cual la función booleana toma el valor 1, es decir, cada fila de la tabla de verdad donde la salida es verdadera se identifica con un número decimal que corresponde a la interpretación binaria de los valores de las variables. Por ejemplo, en una función de 3 variables (A, B, C), el mintermino 5 corresponde a la combinación A=1, B=0, C=1 porque 101 en binario es 5 en decimal, y cada mintermino representa un término producto que incluye todas las variables, cada una en su forma normal o negada según su valor en esa combinación. Dos expresiones booleanas son equivalentes si y solo si producen exactamente la misma salida para todas las combinaciones posibles de entrada, lo cual se verifica comparando sus tablas de verdad, porque la tabla de verdad es la representación completa y única del comportamiento de cualquier función booleana. Si dos expresiones tienen la misma tabla de verdad, entonces son funcionalmente idénticas y se puede utilizar cualquiera de ellas para implementar el mismo circuito, permitiendo elegir siempre la más simple para ahorrar componentes y reducir costos.

9. Shannon: medir información en un mensaje

1. ¿Qué problema resuelve el programa?

El programa calcula la entropía de Shannon de un texto, que es una medida de cuánta información contiene una fuente o cuán impredecible es. Esto permite cuantificar la incertidumbre de un mensaje y determinar si es repetitivo y predecible o variado y con mucha información. El programa también permite comparar dos textos distintos para determinar cuál tiene mayor entropía, lo cual es útil para analizar la cantidad de información en diferentes tipos de mensajes o para identificar patrones en textos.

2. ¿Qué idea matemática usa?

El programa utiliza la fórmula de entropía de Shannon $H = -\sum p_i * \log_2(p_i)$, donde p_i es la probabilidad de cada símbolo en el texto, calculada como la frecuencia del símbolo dividida por el total de caracteres. La entropía mide el nivel de incertidumbre o sorpresa asociado a los símbolos de un mensaje, y mientras más variados y equitativos sean los símbolos, mayor será la entropía, mientras que si hay pocos símbolos distintos o uno domina sobre los demás, la entropía será baja. El programa también calcula la entropía máxima posible para el texto, que es $\log_2(N)$ donde N es el número de símbolos distintos, y muestra qué porcentaje de esa entropía máxima se alcanza, lo que permite entender qué tan cerca está el texto de ser completamente aleatorio.

3. ¿Cómo se ejecuta?

El programa usa la biblioteca math para la función $\log_2$, pero al ser parte de la biblioteca estándar de Python no necesita instalación adicional, así que basta con ejecutar el archivo

"9_Shannon.py" en cualquier editor de código, elegir entre las opciones de calcular la entropía de un solo texto o comparar dos textos, ingresar el o los textos correspondientes, y el programa mostrará las frecuencias, probabilidades, la entropía calculada, la entropía máxima posible y una interpretación del resultado.

4. ¿Qué pruebas hicieron?

Prueba de cálculo de entropía del texto “¡Buenas noches profesor! :)”. El programa toma en cuenta espacios, mayúsculas y símbolos al calcular la entropía y concluye que el texto tiene alta entropía, y por ende, es variado e impredecible.

```

.*+° ¡Bienvenido al programa de Entropía de Shannon! °+*.

|| La entropía de Shannon mide cuánta información contiene una fuente. ||
|| - Un texto muy repetitivo tiene menos incertidumbre. ||
|| - Se usa la frecuencia y probabilidad de cada símbolo ||

=====
Por favor elija lo que desea hacer:
1. Calcular entropía de un texto
2. Comparar la entropía entre dos textos
3. Salir del programa
=====

Ingrese el número de la opción (1, 2 o 3): 1

.*+° Cálculo de Entropía °+*.

Por favor ingrese el texto: ¡Buenas noches profesor!:)

-----
Símbolo | Frecuencia | Probabilidad
-----
i | 1 | 3.8462%
B | 1 | 3.8462%
u | 1 | 3.8462%
e | 3 | 11.5385%
n | 2 | 7.6923%
a | 1 | 3.8462%
s | 3 | 11.5385%
 | 2 | 7.6923%
o | 3 | 11.5385%
c | 1 | 3.8462%
h | 1 | 3.8462%
p | 1 | 3.8462%
r | 2 | 7.6923%
f | 1 | 3.8462%
! | 1 | 3.8462%
: | 1 | 3.8462%
) | 1 | 3.8462%
-----

Longitud del texto: 26 caracteres
Cantidad de símbolos distintos: 17
Entropía de Shannon: 3.9210 bits

-----
El texto tiene alta entropía -> Muy variado y e impredecible.
-----

```

Prueba de comparación de textos, entre uno repetitivo compuesto por 0 y 1 y “Este es un mensaje más variado”. El programa analiza el largo de cada texto y los símbolos distintos de cada uno, concluyendo que el segundo texto tiene mayor entropía, comparando explícitamente la entropía de ambos, por lo que este es más variado y contiene más información.

```
Ingrese el número de la opción (1, 2 o 3): 2

.*+° Comparación de Entropía °+*.

texto 1:
Ingrese el primer texto: 1010101010

texto 2:
Ingrese el segundo texto: Este es un mensaje más variado

-----
Comparación de textos:
-----

texto 1: '1010101010'
- Longitud: 10 caracteres
- Símbolos distintos: 2
- Entropía: 1.0000 bits

texto 2: 'Este es un mensaje más variado'
- Longitud: 30 caracteres
- Símbolos distintos: 16
- Entropía: 3.6947 bits

-----
El texto 2 tiene mayor entropía (3.6947 > 1.0000)
-> El texto 2 es más variado y contiene más información.
-----
```

Prueba de comparación de entropía entre dos textos iguales. El programa concluye que ambos textos tienen la misma entropía, confirmando el funcionamiento correcto del mismo en el cálculo de entropía y en la comparación para casos idénticos.

```
Ingrese el número de la opción (1, 2 o 3): 2
```

```
.*+º Comparación de Entropía º+*.
```

```
texto 1:
```

```
Ingrese el primer texto: Textos iguales
```

```
texto 2:
```

```
Ingrese el segundo texto: Textos iguales
```

```
-----  
Comparación de textos:  
-----
```

```
texto 1: 'Textos iguales'
```

- Longitud: 14 caracteres
- Símbolos distintos: 12
- Entropía: 3.5216 bits

```
texto 2: 'Textos iguales'
```

- Longitud: 14 caracteres
- Símbolos distintos: 12
- Entropía: 3.5216 bits

```
-----  
Ambos textos tienen la misma entropía (3.5216)
```

```
-> Contienen la misma cantidad de información.  
-----
```

Prueba de calcular la entropía de un texto con un mismo símbolo repetido “!!!”, el programa devuelve que el texto no posee entropía correctamente, ya que la probabilidad de encontrar el símbolo ‘!’ en todas las posiciones es de 100%.

```

Ingrese el número de la opción (1, 2 o 3): 1
.*+° Cálculo de Entropía °+*.

Por favor ingrese el texto: !!!

-----
Símbolo | Frecuencia | Probabilidad
-----
!      | 3          | 100.0000%
-----

Longitud del texto: 3 caracteres
Cantidad de símbolos distintos: 1
Entropía de Shannon: 0.0000 bits

-----
El texto no tiene entropía -> Sin información nueva.
-----

```

5. ¿Qué limitaciones tiene la solución?

El programa considera cada carácter individual como un símbolo, incluyendo espacios, signos de puntuación y cualquier otro carácter visible o invisible, pero no hace distinción entre mayúsculas y minúsculas, lo que significa que 'H' y 'h' se tratan como caracteres diferentes. La entropía máxima depende directamente del número de símbolos distintos presentes en el texto, y para textos muy cortos la entropía puede no ser tan significativa estadísticamente.

6. Expliquen en palabras sencillas por qué la entropía mide incertidumbre y no simplemente longitud del texto.

La entropía no mide cuánto texto hay, sino qué tan variado es y qué tanta información nueva contiene, porque un texto largo pero muy repetitivo como "aaaaaaaaaa" tiene entropía cero a pesar de tener 10 caracteres, ya que sabiendo el primer carácter se pueden predecir todos los demás y no hay información nueva en cada posición, mientras que un texto más corto pero variado como "hola" tiene entropía positiva porque hay varios símbolos distintos y no se puede predecir fácilmente el siguiente carácter. En otras palabras, la entropía mide la incertidumbre promedio de cada símbolo: si todos los símbolos son iguales no hay incertidumbre y la entropía es baja o cero, pero si hay muchos símbolos distintos y aparecen con frecuencias similares, la incertidumbre es alta y la entropía también lo es, y por eso un mensaje de pocos caracteres puede tener más entropía que otro mucho más largo pero muy repetitivo. La longitud del texto solo afecta la entropía en la medida en que permite tener más símbolos distintos o distribuciones de probabilidad más variadas, pero el valor de la entropía no depende del largo del mensaje sino de la variedad y frecuencia de los símbolos que lo componen.

10. Primer simulador cuántico: bits, qubits y mediciones

1. ¿Qué problema resuelve el programa?

El programa resuelve el problema de simular el comportamiento básico de un qubit, que es la unidad fundamental de información en computación cuántica, permitiendo aplicar compuertas cuánticas X, Z y H al estado inicial $|0\rangle$ y observar cómo se transforma el estado del qubit. Esto permite entender conceptos abstractos como la superposición cuántica y las probabilidades de medición sin necesidad de tener acceso a un computador cuántico real, haciendo tangible el estudio de la computación cuántica desde un entorno de programación convencional.

2. ¿Qué idea matemática usa?

El programa utiliza el formalismo de la mecánica cuántica para sistemas de dos niveles, representando el estado de un qubit como un vector de dos componentes $[\alpha, \beta]$ donde α y β son números que cumplen $|\alpha|^2 + |\beta|^2 = 1$. Las compuertas cuánticas se representan como matrices de 2×2 que actúan sobre este vector: la compuerta $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ intercambia α y β , la compuerta $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ cambia el signo de β , y la compuerta $H = (1/\sqrt{2})\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ crea superposición al transformar $|0\rangle$ en $(|0\rangle + |1\rangle)/\sqrt{2}$. Las probabilidades de medir 0 y 1 se calculan como $|\alpha|^2$ y $|\beta|^2$ respectivamente, y la simulación de mediciones utiliza estas probabilidades para generar resultados aleatorios.

3. ¿Cómo se ejecuta?

Se necesita un dispositivo con Python instalado y un editor de código que pueda ejecutar archivos en este lenguaje. El programa presenta un menú interactivo que permite al usuario elegir entre aplicar compuertas individuales (X, Z o H) o una secuencia personalizada de compuertas, mostrando en todo momento el estado del qubit antes y después de cada operación, las probabilidades de medición y los resultados de una simulación de 1000 mediciones.

4. ¿Qué pruebas hicieron?

Se realizaron múltiples pruebas para verificar el correcto funcionamiento del simulador cuántico. Se comprobó la compuerta X intercambiando correctamente los coeficientes α y β (verificando que $X|0\rangle = |1\rangle$), la compuerta H creando superposición con probabilidades del 50% para $|0\rangle$ y $|1\rangle$, y la propiedad $HH|0\rangle = |0\rangle$ con errores numéricos mínimos. También se verificó la compuerta Z cambiando el signo de β , la normalización automática de estados no normalizados, y la simulación de mediciones con diferentes cantidades (100, 1000, 10000) confirmando que las frecuencias convergen a las probabilidades teóricas. Adicionalmente, se probaron múltiples estados cuánticos incluyendo estados base, superposiciones reales y complejas, y estados con probabilidades asimétricas (75%/25%, 90%/10%), así como el manejo de errores con entradas inválidas, confirmando que el programa funciona correctamente en todos los casos.


```
Ingrese el estado inicial del qubit:
El estado se representa como  $\alpha|0\rangle + \beta|1\rangle$ , donde  $|\alpha|^2 + |\beta|^2 = 1$ 
Ingrese la parte real de  $\alpha$  (coeficiente de  $|0\rangle$ ): 1
Ingrese la parte imaginaria de  $\alpha$  (coeficiente de  $|0\rangle$ ): 0
Ingrese la parte real de  $\beta$  (coeficiente de  $|1\rangle$ ): 0
Ingrese la parte imaginaria de  $\beta$  (coeficiente de  $|1\rangle$ ): 0

Estado inicial del qubit:
Estado: (1.0000+0.0000j)|0> + (0.0000+0.0000j)|1>
Probabilidad de medir 0: 1.0000 (100.00%)
Probabilidad de medir 1: 0.0000 (0.00%)

-----

Compuertas disponibles:
1. X (NOT cuantica)
2. Z (Fase)
3. H (Hadamard)
4. Mostrar estado actual
5. Simular mediciones
6. Salir y terminar programa

Seleccione una opcion (1-6): 3

Despues de aplicar compuerta H:
Estado: (0.7071+0.0000j)|0> + (0.7071+0.0000j)|1>
Probabilidad de medir 0: 0.5000 (50.00%)
Probabilidad de medir 1: 0.5000 (50.00%)

-----

Compuertas disponibles:
1. X (NOT cuantica)
2. Z (Fase)
3. H (Hadamard)
4. Mostrar estado actual
5. Simular mediciones
6. Salir y terminar programa

Seleccione una opcion (1-6): 6

Saliendo del simulador...
```

```
Ingrese el estado inicial del qubit:  
El estado se representa como  $\alpha|0\rangle + \beta|1\rangle$ , donde  $|\alpha|^2 + |\beta|^2 = 1$   
Ingrese la parte real de  $\alpha$  (coeficiente de  $|0\rangle$ ): 0.3  
Ingrese la parte imaginaria de  $\alpha$  (coeficiente de  $|0\rangle$ ): 0.4  
Ingrese la parte real de  $\beta$  (coeficiente de  $|1\rangle$ ): 0.5  
Ingrese la parte imaginaria de  $\beta$  (coeficiente de  $|1\rangle$ ): -0.6
```

```
Advertencia: El estado no esta normalizado ( $|\alpha|^2 + |\beta|^2 = 0.8600$ )  
Se normalizara automaticamente.
```

```
Estado inicial del qubit:  
Estado:  $(0.3235+0.4313j)|0\rangle + (0.5392-0.6470j)|1\rangle$   
Probabilidad de medir 0: 0.2907 (29.07%)  
Probabilidad de medir 1: 0.7093 (70.93%)
```

```
-----  
Compuertas disponibles:
```

1. X (NOT cuantica)
2. Z (Fase)
3. H (Hadamard)
4. Mostrar estado actual
5. Simular mediciones
6. Salir y terminar programa

```
Seleccione una opcion (1-6): 1
```

```
Despues de aplicar compuerta X:  
Estado:  $(0.5392-0.6470j)|0\rangle + (0.3235+0.4313j)|1\rangle$   
Probabilidad de medir 0: 0.7093 (70.93%)  
Probabilidad de medir 1: 0.2907 (29.07%)
```

```
-----  
Compuertas disponibles:
```

1. X (NOT cuantica)
2. Z (Fase)
3. H (Hadamard)
4. Mostrar estado actual
5. Simular mediciones
6. Salir y terminar programa

```
Seleccione una opcion (1-6): 6
```

```
Saliendo del simulador...
```

```

Ingrese el estado inicial del qubit:
El estado se representa como  $\alpha|0\rangle + \beta|1\rangle$ , donde  $|\alpha|^2 + |\beta|^2 = 1$ 
Ingrese la parte real de  $\alpha$  (coeficiente de  $|0\rangle$ ): 0.5
Ingrese la parte imaginaria de  $\alpha$  (coeficiente de  $|0\rangle$ ): 0.86
Ingrese la parte real de  $\beta$  (coeficiente de  $|1\rangle$ ): 0
Ingrese la parte imaginaria de  $\beta$  (coeficiente de  $|1\rangle$ ): 0

Advertencia: El estado no esta normalizado ( $|\alpha|^2 + |\beta|^2 = 0.9896$ )
Se normalizara automaticamente.

Estado inicial del qubit:
Estado:  $(0.5026+0.8645j)|0\rangle + (0.0000+0.0000j)|1\rangle$ 
Probabilidad de medir 0: 1.0000 (100.00%)
Probabilidad de medir 1: 0.0000 (0.00%)

-----

Compuertas disponibles:
1. X (NOT cuantica)
2. Z (Fase)
3. H (Hadamard)
4. Mostrar estado actual
5. Simular mediciones
6. Salir y terminar programa

Seleccione una opcion (1-6): 2

Despues de aplicar compuerta Z:
Estado:  $(0.5026+0.8645j)|0\rangle + (-0.0000-0.0000j)|1\rangle$ 
Probabilidad de medir 0: 1.0000 (100.00%)
Probabilidad de medir 1: 0.0000 (0.00%)

-----

Compuertas disponibles:
1. X (NOT cuantica)
2. Z (Fase)
3. H (Hadamard)
4. Mostrar estado actual
5. Simular mediciones
6. Salir y terminar programa

Seleccione una opcion (1-6): 6

Saliendo del simulador...

```

5. ¿Qué limitaciones tiene la solución?

La principal limitación es que el programa simula únicamente un qubit, mientras que los computadores cuánticos reales trabajan con múltiples qubits y fenómenos como el entrelazamiento que no están representados aquí. La simulación de mediciones utiliza números pseudoaleatorios para imitar el comportamiento cuántico, pero no es una ejecución en un computador cuántico real, por lo que no se incluyen efectos físicos como la decoherencia o los errores cuánticos. Además, el programa usa números reales en lugar de números complejos, lo cual es suficiente para este nivel educativo pero no para simulaciones cuánticas más avanzadas.

6. Expliquen la diferencia entre probabilidad cuántica simulada y la ejecución en un computador cuántico real.

En el simulador, las probabilidades se calculan matemáticamente usando la fórmula $|\alpha|^2$ y $|\beta|^2$ y las mediciones se simulan generando números aleatorios con esas probabilidades, lo cual es determinista en el sentido de que siempre se pueden calcular exactamente las probabilidades esperadas antes de medir. En un computador cuántico real, el estado cuántico existe realmente como una superposición física y la medición colapsa el estado a un valor con una probabilidad dada por la naturaleza cuántica, que incluye efectos como la decoherencia, el ruido y las imperfecciones de las compuertas que pueden alterar las probabilidades teóricas. Además, en un computador real, el proceso de medición es físico y no se puede simular perfectamente con números aleatorios, porque la aleatoriedad cuántica es fundamental y no determinista, mientras que en el simulador los números aleatorios son generados por un algoritmo pseudoaleatorio que eventualmente es determinista. En resumen, el simulador calcula lo que debería pasar en teoría, mientras que un computador real ejecuta lo que realmente pasa en la naturaleza cuántica, con todas sus imperfecciones y su aleatoriedad genuina.